SensiSpace

Technology Stack

Detailed Explanation of Technologies Used

Stack Overview:

```
Frontend:  HTML5 + CSS3 + JavaScript (Vanilla)
Backend:   Python 3.9 + FastAPI + Uvicorn
ML:        PyTorch + torchvision
Vision:    OpenCV + Pillow (PIL)
Data:      NumPy
```

1. Python 3.9

Python is a high-level, interpreted programming language known for its readability and extensive ecosystem. It is the dominant language in machine learning and data science due to its rich library support.

Why Python?

Reason	Detail
ML ecosystem	PyTorch, TensorFlow, scikit-learn - all Python-first
Rapid prototyping	Write and test code quickly without compilation
Community	Largest ML community, extensive documentation
Library support	400,000+ packages on PyPI for any task

How We Use It

- All backend code: model definitions, training, API, visualization
- Entry points: train.py (training), app.py (server), gradcam.py (visualization)
- Manages entire ML pipeline: data loading, preprocessing, inference, encoding

2. PyTorch

PyTorch is an open-source deep learning framework developed by Meta AI Research. It provides tensor computation with GPU acceleration, automatic differentiation (autograd) for training neural networks, and dynamic computational graphs for flexible model design.

Why PyTorch over TensorFlow?

Feature	PyTorch	TensorFlow
Graph type	Dynamic (eager)	Static (graph mode)
Debugging	Easy - standard Python	Harder - graph execution
Research use	80%+ of papers	Declining in research
Learning curve	Pythonic, intuitive	Steeper, more boilerplate
Apple Silicon	Native MPS support	Limited

How We Use It

Model Definition (nn.Module)

```
class BabyDragonHatchling(nn.Module):
    def __init__(self):
        self.stage1 = BDHStage(64, 64)
        self.stage2 = BDHStage(64, 128)
    def forward(self, x):
        x = self.stage1(x)
        return x
```

Every model component (MultiScaleBlock, LateralInhibition, ContextualAttention) is a nn.Module subclass with learnable parameters.

Key PyTorch Components Used

Component	Purpose in SensiSpace
nn.Conv2d	Convolutional layers in all model stages
nn.BatchNorm2d	Normalizes activations for stable training
nn.Linear	Fully connected classifier layers
nn.MaxPool2d	Downsamples spatial dimensions between stages
nn.AdaptiveAvgPool2d	Global average pooling before classifier
nn.Dropout	Regularization to prevent overfitting
nn.Parameter	Learnable alpha in lateral inhibition
F.relu / F.softmax	Non-linear activation / probability conversion
optim.Adam	Optimizer for training both models

CrossEntropyLoss	Classification loss function

3. torchvision

A PyTorch companion library providing pre-trained models (ResNet, VGG, Inception), image transformations (resize, normalize, augment), and dataset utilities (ImageFolder for loading labeled images).

How We Use It

Pre-trained ResNet-18

```
from torchvision.models import resnet18, ResNet18_Weights
model = resnet18(weights=ResNet18_Weights.IMAGENET1K_V1)
model.fc = nn.Linear(512, 10)  # Replace for 10 classes
```

This gives us a model pre-trained on 1.3 million ImageNet images - it already knows edges, textures, and shapes. We only fine-tune it on satellite imagery.

Image Transforms

```
transforms.Compose([
    transforms.Resize((64, 64)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(15),
    transforms.ColorJitter(brightness=0.2),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                         std=[0.229, 0.224, 0.225])
])
```

Dataset Loading

```
dataset = ImageFolder("data/eurosat/2750/", transform=transform)
loader = DataLoader(dataset, batch_size=64, shuffle=True)
```

ImageFolder automatically maps folder names to class labels.

4. FastAPI

FastAPI is a modern, high-performance Python web framework for building APIs. Built on Starlette (async) and Pydantic (validation). It is one of the fastest Python web frameworks available.

Why FastAPI over Flask/Django?

Feature	FastAPI	Flask	Django
Speed	Very fast (async)	Medium	Slower
Auto docs	Swagger auto-gen	Manual	Manual
Type safety	Pydantic	None	Forms-based
Async	Native	Plugin	Limited
Simplicity	Minimal code	Minimal	Heavy

API Endpoints

Endpoint	Method	Returns
/api/predict	POST	Both model predictions + spectral overlay
/api/sample	GET	Random EuroSAT sample image
/api/metrics	GET	Training benchmark data (accuracy, F1, etc.)
/api/colors	GET	Color palette for all 10 land cover classes

Code Example

```
@app.post("/api/predict")
async def predict(file: UploadFile):
    image = Image.open(io.BytesIO(await file.read()))
    resnet_result = classify(resnet_model, image)
    bdh_result = classify(bdh_model, image)
    return {"resnet": resnet_result, "baby_dragon": bdh_result}
```

5. Uvicorn

Uvicorn is an ASGI (Asynchronous Server Gateway Interface) web server for Python. It runs FastAPI applications and handles HTTP requests efficiently using async I/O.

- Recommended by FastAPI as the default server
- Async I/O - handles concurrent requests efficiently
- Hot reload - auto-restarts on code changes during development
- Production-ready - can be paired with Gunicorn for multi-worker deployment

```
uvicorn app:app --host 0.0.0.0 --port 8000
```

6. OpenCV (cv2)

OpenCV (Open Source Computer Vision Library) is the world's most used computer vision library with 2,500+ algorithms for image processing, object detection, and video analysis.

How We Use It

Spectral Segmentation

```
hsv = cv2.cvtColor(img, cv2.COLOR_RGB2HSV)  # Color space
lab = cv2.cvtColor(img, cv2.COLOR_RGB2LAB)   # For urban detection
```

Contour Detection

```
contours, _ = cv2.findContours(mask,
    cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
cv2.drawContours(overlay, contours, -1, color, 2)
```

Morphological Operations

```
kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5,5))
mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, kernel)  # Fill
mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel)   # Clean
```

Key Functions Used

Function	Purpose
cvtColor	Convert RGB to HSV/LAB for spectral analysis
findContours	Detect land cover region boundaries
drawContours	Draw colored borders on overlay
morphologyEx	Clean masks (close gaps, remove noise)
countNonZero	Count pixels in a mask for area %
moments	Calculate centroid for label placement

7. Pillow (PIL)

Pillow is the Python Imaging Library - the standard for opening, manipulating, and saving image files in Python.

How We Use It

```
# Open uploaded image
image = Image.open(io.BytesIO(file_bytes))

# Draw text labels on overlay
draw = ImageDraw.Draw(result)
draw.text((x, y), "Forest 95.8%", fill=(34,220,77))

# Encode to base64 for API response
buffer = io.BytesIO()
image.save(buffer, format="JPEG", quality=92)
b64 = base64.b64encode(buffer.read()).decode("utf-8")
```

Function	Purpose
Image.open()	Load uploaded satellite image
image.resize()	Scale to display size (400x400)
ImageDraw	Draw class labels and badges on overlay
ImageFont	Load system fonts for text rendering
image.save()	Encode result as JPEG for API response

8. NumPy

NumPy is the fundamental package for numerical computing in Python. It provides N-dimensional arrays and mathematical operations that are 50-100x faster than pure Python loops.

How We Use It

```
# Convert PIL image to array
img = np.array(image, dtype=np.uint8)

# Compute Green Leaf Index (vegetation)
r, g, b = img[:,:,0], img[:,:,1], img[:,:,2]
gli = (2*g - r - b) / (2*g + r + b + 1)

# Boolean mask for vegetation pixels
veg_mask = (gli > 0.05) & (s > 40) & (h >= 25) & (h <= 90)
```

NumPy enables vectorized pixel operations - processing all 160,000 pixels (400x400) simultaneously instead of looping through each one.

9. HTML5 + CSS3 + JavaScript

Why Vanilla (No React/Vue)?

Reason	Detail
Simplicity	No build tools, no npm, no bundling. Just 3 files.
Speed	Zero framework overhead. Instant page load.
Portability	Works in any browser. No Node.js dependency.
Integration	Served directly by FastAPI as static files.

HTML5 (index.html)

- Semantic structure: <header>, <main>, <section>
- Tab-based navigation: ResNet / BDH / Comparison
- Image upload via <input type="file"> with drag-and-drop
- SEO-optimized with meta tags and proper heading hierarchy

CSS3 (style.css)

Feature	Purpose
Custom Properties	Design tokens (colors, spacing, fonts)
Flexbox & Grid	Responsive layout for cards and tabs
Glassmorphism	backdrop-filter: blur() for premium look
CSS Animations	Background orbs, hover effects, loading states
Dark mode	Full dark-mode palette with high-contrast colors
Media queries	Responsive on mobile, tablet, desktop

JavaScript (app.js)

Feature	What it does
fetch() API	Calls FastAPI endpoints (/predict, /sample)
FormData	Uploads image file to server
Dynamic DOM	Creates result cards and charts programmatically
Tab switching	Shows/hides ResNet, BDH, Comparison panels
Color-coded bars	Probability distribution charts
Base64 rendering	Displays spectral overlay images

10. Summary

Technology	Version	Role	Why Chosen
Python	3.9	Core language	ML ecosystem, readability
PyTorch	2.x	Deep learning	Dynamic graphs, MPS support
torchvision	0.x	Models + transforms	Pretrained ResNet, data load
FastAPI	0.100+	Web backend	Fast, async, auto-docs
Uvicorn	0.x	ASGI server	Async I/O, hot reload
OpenCV	4.x	Computer vision	Spectral segmentation
Pillow	10.x	Image I/O	Load, draw, encode images
NumPy	1.x	Numerical compute	Vectorized pixel ops
HTML/CSS/JS	5/3/ES6	Frontend	No deps, instant load
fpdf2	2.x	PDF generation	Report and docs